

《系统开发工具基础》实验报告



中国海洋大学
OCEAN UNIVERSITY OF CHINA

学号： 24120011027

姓名： 严程漪

专业： 物流管理

我的 github: [点击这里](#)

2026 年 9 月 10 日

目录

1	实验目的	2
1.1	了解命令行环境	2
1.2	了解学习开发环境与工具	2
1.3	学习 Debugging and Profil	2
2	课内实验	2
2.1	第一题：控制一个可清理的后台任务	2
2.2	第二题：语义重构与本地开发反馈	3
2.3	第三题：用调试器定位归并排序缺陷	5
2.4	第四题：先测量，再优化慢速词频程序	6
3	课外实例	8
3.1	实例一：-参数机制	8
3.2	实例二：marco 和 polo 函数	8
3.3	实例三：重复运行脚本	9
3.4	实例四：安装 IDE 扩展	10
3.5	实例五：分离偶数和奇数	10
3.6	实例六：内存错误调试	11
3.7	实例七：使用 strace 跟踪系统调用	12
3.8	实例八：通过 perf 对 C 语言采样并绘制火焰图	12
3.9	实例九：htop 监视 +taskset 绑定 CPU	14
3.10	实例十：修改 SSH 配置	16
4	解题感悟	18

1 实验目的

1.1 了解命令行环境

1.2 了解学习开发环境与工具

1.3 学习 Debugging and Profil

2 课内实验

2.1 第一题：控制一个可清理的后台任务

1、赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log 先写入脚本，再赋予脚本执行权限

```
ouc@islouc-vm:~/q01/work/24120011027$ mkdir -p q05 && cd q05
ouc@islouc-vm:~/q01/work/24120011027/q05$ cat << 'EOF' > worker.sh
> trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
> n=0
> while true; do
>   echo "$n"
>   n=$((n+1))
>   sleep 1
> done
> EOF
ouc@islouc-vm:~/q01/work/24120011027/q05$ chmod +x worker.sh
```

图 1: worker.sh 脚本

再将标准输出和标准错误写入

```
ouc@islouc-vm:~/q01/work/24120011027/q05$ ./worker.sh > stdout.log 2> stderr.log
```

图 2: 写入标准输出和标准错误

2、使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。

```
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
ouc@islouc-vm:~/q01/work/24120011027/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
ouc@islouc-vm:~/q01/work/24120011027/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
```

图 3: 运行状态

& 代表后台运行的标志

3、使用 jobs -p 或等价方式取得 PID，不得手工抄写 PID；发送 SIGTERM 并等待任务结束

```
ouc@islouc-vm:~/q01/work/24120011027/q05$ kill -TERM $(jobs -p)
ouc@islouc-vm:~/q01/work/24120011027/q05$ sleep 2
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
```

图 4: 获取 PID 并发送终止信号

Kill: 用来向指定进程发送信号的命令; -TERM: 发送 SIGTERM (终止信号) \$(...): 命令替换语法, 先执行括号里的语法, 把得到的 PID 数字“挤”回原位置, 避免人工手抄;
Job -p: 输出当前终端后台任务的进程 PID

4、确认 cleanup.log 中出现 CLEAN_EXIT, 且任务已经退出; 禁止使用 kill -9

```
ouc@islouc-vm:~/q01/work/24120011027/q05$ cat cleanup.log
CLEAN_EXIT
ouc@islouc-vm:~/q01/work/24120011027/q05$ jobs
ouc@islouc-vm:~/q01/work/24120011027/q05$
```

图 5: 状态确认

2.2 第二题: 语义重构与本地开发反馈

1、在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06, 并启用 Python 语言服务器先创建三个 Python 文件

```
ouc@islouc-vm:~$ mkdir -p q06 && cd q06
ouc@islouc-vm:~/q06$ cat << 'EOF' > math_utils.py
> def total_price(price: float, count: int) -> float:
>     return price * count
> EOF
ouc@islouc-vm:~/q06$ cat << 'EOF' > app.py
> from math_utils import total_price
> print(total_price(12.5, 4))
> EOF
ouc@islouc-vm:~/q06$ cat << 'EOF' > test_math_utils.py
> from math_utils import total_price
> def test_total_price():
>     assert total_price(12.5, 4) == 50.0
> EOF
```

图 6: 文件创建

2、分别演示跳转到定义和查找引用

```
ouc@islouc-vm:~/q06$ grep -rn "def total_price" .
./math_utils.py:1:def total_price(price: float, count: int) -> float:
ouc@islouc-vm:~/q06$ grep -rn "total_price" .
./test_math_utils.py:1:from math_utils import total_price
./test_math_utils.py:2:def test_total_price():
./test_math_utils.py:3:    assert total_price(12.5, 4) == 50.0
./math_utils.py:1:def total_price(price: float, count: int) -> float:
./app.py:1:from math_utils import total_price
./app.py:2:print(total_price(12.5, 4))
```

图 7: 跳转到定义与查找引用

3、使用“重命名符号”把 total_price 改为 calculate_total, 保证定义和所有代码引用同步改变

```
ouc@islouc-vm:~/q06$ sed -i 's/total_price/calculate_total/g' math_utils.py app.py test_math_utils.py
ouc@islouc-vm:~/q06$ grep -rn "calculate_total" .
./test_math_utils.py:1:from math_utils import calculate_total
./test_math_utils.py:2:def test_calculate_total():
./test_math_utils.py:3:    assert calculate_total(12.5, 4) == 50.0
./math_utils.py:1:def calculate_total(price: float, count: int) -> float:
./app.py:1:from math_utils import calculate_total
./app.py:2:print(calculate_total(12.5, 4))
```

图 8: 重命名结果

4、在 app.py 中临时加入未使用的 import os，用 ruff 发现并删除，最后运行 ruff 和 pytest 并保证通过。

```
ouc@islouc-vm:~/q06$ sed -i '1i import os' app.py
ouc@islouc-vm:~/q06$ ruff check .
```

图 9: 代码运行

发现未使用的 os

```
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | / import os
2 | | from math_utils import calculate_total
3 | | print(calculate_total(12.5, 4))
  | ^
help: Organize imports
1 | import os
2 | +
3 | from math_utils import calculate_total
4 | +
5 | print(calculate_total(12.5, 4))
  |
F401 [*] `os` imported but unused
```

图 10: 结果展示

然后删除第一行

```
ouc@islouc-vm:~/q06$ sed -i '1i import os' app.py
```

图 11: 删除命令

运行 ruff 和 pytest 并保证通过

```
ouc@islouc-vm:~/q06$ ruff check .
All checks passed!
```

图 12: 通过截图

```
ouc@islouc-vm:~/q06$ python3 -m pytest
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/q06
collected 1 item

test_math_utils.py . [100%]

===== 1 passed in 0.01 seconds =====
```

图 13: 通过截图

2.3 第三题：用调试器定位归并排序缺陷

1、先运行给定输入，确认结果错误或出现异常

```
ouc@islouc-vm:~/q07$ cat << 'EOF' > merge_sort.py
> def merge(left, right):
>     result = []
>     i = j = 0
>     while i < len(left) and j < len(right):
>         if left[i] <= right[j]:
>             result.append(left[i]); i += 1
>         else:
>             result.append(right[j]); j += 1
>     return result + left[i:] + right[j:]
> def merge_sort(a):
>     if len(a) <= 1: return a
>     m = len(a) // 2
>     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
> print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
> EOF
ouc@islouc-vm:~/q07$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 14: merge_sort.py

原本应该出现的升序列表完全混乱了

2、使用 pdb、IDE debugger 或等价调试器，在 merge 中设置断点并观察 i、j、left[i]、right[j]。

```
ouc@islouc-vm:~/q07$ python3 -m pdb merge_sort.py
> /home/ouc/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b merge
Breakpoint 1 at /home/ouc/q07/merge_sort.py:1
(Pdb) c
> /home/ouc/q07/merge_sort.py(2)merge()
-> result = []
(Pdb) n
> /home/ouc/q07/merge_sort.py(3)merge()
-> i = j = 0
(Pdb) n
> /home/ouc/q07/merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
(Pdb) n
> /home/ouc/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(Pdb) n
> /home/ouc/q07/merge_sort.py(8)merge()
-> result.append(right[j]); j += 1
```

图 15: 断点观察结果

```
(Pdb) p i,j,left[i],right[j]
(0, 0, 3, 1)
```

图 16: 断点观察结果

Left[i] 是 3, right[j] 是 1, 所以走到了 else 分支, 但是原分支中写加入列表的是 right[i], 但实际应该是 right[j] 改过来应该就行了

3、完成最小修复, 不得用 sorted 替换归并排序

```
ouc@islouc-vm:~/q07$ sed -i 's/right\[i\]/right\[j\]/g' merge_sort.py
ouc@islouc-vm:~/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
ouc@islouc-vm:~/q07$
```

图 17: 修复及排序结果

4、使用 pytest 增加两个测试: 给定输入和含重复元素的列表; 保证测试通过

```
ouc@islouc-vm:~/q07$ cat << 'EOF' > test_merge_sort.py
> from merge_sort import merge_sort
>
> def test_given_input():
>     assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
>
> def test_duplicate_elements():
>     assert merge_sort([5, 2, 2, 8, 5, 1, 1]) == [1, 1, 2, 2, 5, 5, 8]
> EOF
ouc@islouc-vm:~/q07$ pytest-3
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/q07
collected 2 items

test_merge_sort.py .. [100%]

===== 2 passed in 0.01 seconds =====
```

图 18: 测试结果

可以看出测试是通过了的

2.4 第四题: 先测量, 再优化慢速词频程序

1、生成 words.txt, 使用 time 运行原程序 2 次并记录耗时。

```
ouc@islouc-vm:~$ mkdir -p q08 && cd q08
ouc@islouc-vm:~/q08$ cat << 'EOF' > generate_words.py
> import random
> random.seed(20260831)
> vocab = [f"w{i}" for i in range(1000)]
> with open("words.txt", "w", encoding="utf-8") as f:
>     f.write(" ".join(random.choice(vocab) for _ in range(30000)))
> EOF
ouc@islouc-vm:~/q08$ cat << 'EOF' > wordfreq.py
> words = open("words.txt", encoding="utf-8").read().split()
> unique = []
> for word in words:
>     if word not in unique:
>         unique.append(word)
> print("count=", len(unique))
> EOF
ouc@islouc-vm:~/q08$ python3 generate_words.py
ouc@islouc-vm:~/q08$ ls words.txt
words.txt
```

图 19: generate_word.py

由图可知, 成功生成了 words.txt 文件

```
ouc@islouc-vm:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.100s
user    0m0.089s
sys     0m0.008s
ouc@islouc-vm:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.103s
user    0m0.095s
sys     0m0.005s
```

图 20: 运行结果截图

运行两次并记录耗时，可以看出一次耗时 0.100s，一次耗时 0.103s。

2、使用 cProfile -s cumulative 运行一次，找出主要耗时位置

```
ouc@islouc-vm:~/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.074 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
    1   0.000    0.000    0.074    0.074 {built-in method builtins.exec}
    1   0.071    0.071    0.074    0.074 wordfreq.py:1(<module>)
    1   0.003    0.003    0.003    0.003 {method 'split' of 'str' objects}
    1   0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
    1   0.000    0.000    0.000    0.000 codecs.py:319(decode)
    1   0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
    1   0.000    0.000    0.000    0.000 {built-in method builtins.print}
1000   0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
    1   0.000    0.000    0.000    0.000 {built-in method io.open}
    1   0.000    0.000    0.000    0.000 codecs.py:309(__init__)
    1   0.000    0.000    0.000    0.000 {built-in method builtins.len}
    1   0.000    0.000    0.000    0.000 codecs.py:260(__init__)
    1   0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

图 21: 耗时截图

可以看出主要耗时的是 for 循环

3、在不改变最终 count 的前提下优化程序；不得写死 1000

```
ouc@islouc-vm:~/q08$ cat << 'EOF' > wordfreq.py
> words = open("words.txt", encoding="utf-8").read().split()
> unique_set = set()
> for word in words:
>     unique_set.add(word)
>
> unique = list(unique_set)
> print("count=", len(unique))
> EOF
```

图 22: wordfreq.py

集合自带有查重功能，所以不用使用 for 循环一次一次遍历

4、使用相同输入再运行 2 次，计算优化后前后中位数和加速比。


```
ouc@islouc-vm:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.033s
user    0m0.016s
sys     0m0.012s
ouc@islouc-vm:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.028s
user    0m0.020s
sys     0m0.008s
```

图 23: 结果截图.py

可以看出时间大大减少优化前中位数为: $(0.100+0.103) / 2=0.1015$

优化后中位数为: $(0.033+0.028) / 2=0.0305$

加速比为: $0.1015/0.0305=3.328$

3 课外实例

3.1 实例一: -参数机制

- 1、-可以处理以-开头的文件或目录名，避免命令把文件名解析为命令行参数。
- 2、传递带有-的参数给后续子命令或位置参数。
- 3、运行 `touch - -myfile`

```
ouc@islouc-vm:~$ touch - -myfile
ouc@islouc-vm:~$ ls -l -- -myfile
-rw-rw-r-- 1 ouc ouc 0 9月  6 21:56 -myfile
```

图 24: 运行截图

- 4、在不使用-的情况下把它删掉。

```
ouc@islouc-vm:~$ rm ./-myfile
ouc@islouc-vm:~$ ls -l -- -myfile
ls: cannot access '-myfile': No such file or directory
```

图 25: 不使用-删除截图

3.2 实例二: marco 和 polo 函数

- 1、编写 marco.sh 函数

```
ouc@islouc-vm:~$ cat << 'EOF' > marco.sh
> #!/bin/bash
> marco() {
>     export MARCO_LOCATION=$(pwd)
>     echo "Saved location: $MARCO_LOCATION"
> }
>
> if [ -z "$MARCO_LOCATION" ]; then
>     echo "Error: No location saved yet. Run 'marco' first."
> else
>     cd "$MARCO_LOCATION" || return
>     echo "Returned to: $(pwd)"
> fi
> }
> EOF
```

图 26: marco.sh

2、运行结果

```
ouc@islouc-vm:~$ source marco.sh
ouc@islouc-vm:~$ cd /tmp && marco
Saved location: /tmp
ouc@islouc-vm:/tmp$ cd ~
ouc@islouc-vm:~$ pwd
/home/ouc
ouc@islouc-vm:~$ polo
Returned to: /tmp
ouc@islouc-vm:/tmp$ pwd
/tmp
```

图 27: 结果截图

3.3 实例三：重复运行脚本

1、首先，先编写一个 test.sh 文件，这个为佳不仅可以将结果打印出来，并且可以报告它运行了多少次才失败。

```
ouc@islouc-vm:/tmp$ cat << 'EOF' > test.sh
> #!/usr/bin/env bash
> count=0
> while true; do
>     ((count++))
>     ./target.sh > stdout.log 2> stderr.log
>     if [[ $? -ne 0 ]]; then
>         echo "脚本在第 $count 次运行后失败！"
>         cat stdout.log
>         echo "===== stderr ====="
>         cat stderr.log
>         break
>     fi
> done
> EOF
```

图 28: test.sh

2、将 bash 脚本输入

```
ouc@islouc-vm:/tmp$ cat << 'EOF'> target.sh
> #!/usr/bin/env bash
> n=$(( RANDOM % 100 ))
> if [[ n -eq 42 ]]; then
>     echo "Something went wrong"
>     >&2 echo "The error was using magic numbers"
>     exit 1
> fi
> echo "Everything went according to plan"
> EOF
ouc@islouc-vm:/tmp$ chmod +x target.sh
```

图 29: target.sh

3、输出结果

```
ouc@islouc-vm:/tmp$ ./test.sh
脚本在第 68 次运行后失败！
Something went wrong
===== stderr =====
The error was using magic numbers
ouc@islouc-vm:/tmp$
```

图 30: 结果截图

3.4 实例四：安装 IDE 扩展

1、在虚拟机中打开 VS Code，再打开扩展面板，选择并安装一个扩展，这里我选择了 C/C++。

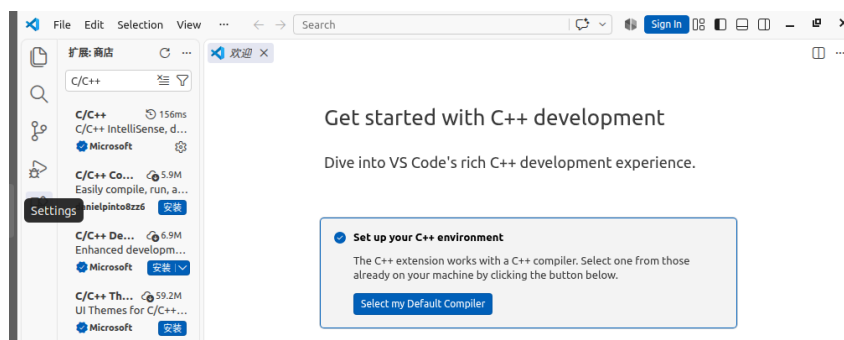


图 31: 结果截图

3.5 实例五：分离偶数和奇数

- 1、创建一个包含 0 到 99 的测试文件 test.txt 并用 Vim 打开
- 2、进入 Vim 后，执行如下操作，在下方条栏执行：
:%d (删除原有的全部截图)；
:put=join(range(0,98,2)) (生成偶数行)；
:put=join(range(1,99,2)) (生成奇数行)；
kkddzz (清理多余空行)
- 3、执行 cat test.txt，检查结果是否正确。

```
ouc@islouc-vm:/tmp$ seq 0 99 > test.txt && vim test.txt
ouc@islouc-vm:/tmp$ cat test.txt
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56
58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94 96 98
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57
59 61 63 65 67 69 71 73 75 77 79 81 83 85 87 89 91 93 95 97 99
```

图 32: 结果截图

3.6 实例六：内存错误调试

1、创建 uaf.c 文件

```
ouc@islouc-vm:/tmp$ cat << 'EOF' > uaf.c
> #include <stdlib.h>
> #include <string.h>
> #include <stdio.h>
>
> int main() {
>     char *greeting = malloc(32);
>     strcpy(greeting, "Hello, world!");
>     printf("%s\n", greeting);
>
>     free(greeting);
>
>     greeting[0] = 'J';
>     printf("%s\n", greeting);
>
>     return 0;
> }
> EOF
```

图 33: uaf.c

2、直接使用 GCC 编译并运行

```
ouc@islouc-vm:/tmp$ gcc uaf.c -o uaf && ./uaf
Hello, world!
J
```

图 34: 运行结果

3、使用 AddressSanitizer 编译运行

```
ouc@islouc-vm:/tmp$ gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
Hello, world!
=====
==7259==ERROR: AddressSanitizer: heap-use-after-free on address 0x603000000010 at
pc 0x557991b642c7 bp 0x7ffdaa7c1940 sp 0x7ffdaa7c1930
WRITE of size 1 at 0x603000000010 thread T0
#0 0x557991b642c6 in main /tmp/uaf.c:12
#1 0x7fa74039d082 in __libc_start_main ../csu/libc-start.c:308
#2 0x557991b6418d in _start (/tmp/uaf+0x118d)
Burp Suite Community Edition
0x603000000010 is located 0 bytes inside of 32-byte region [0x603000000010,0x60300
0000030)
freed by thread T0 here:
```

图 35: 运行结果

由截图可以发现，ASan 发现程序在第 10 行释放堆内存后，又在第 12 行非法访问并修改该已释放内存的安全隐患，并强制终止了程序运行。

4、针对发现的问题，修改 uaf.c 文件

```
ouc@islouc-vm:/tmp$ cat <<'EOF'>uaf.c
> #include <stdlib.h>
> #include <string.h>
> #include <stdio.h>
> int main() {
>     char *greeting = malloc(32);
>     if (greeting == NULL) return 1;
>     strcpy(greeting, "Hello, world!");
>     printf("%s\n", greeting);
>     greeting[0] = 'J';
>     printf("%s\n", greeting);
>     free(greeting);
>     greeting = NULL;
>     return 0;
> }
> EOF
```

图 36: 修改后的 uaf.c

5、运行修改后的文件，得到结果。

```
ouc@islouc-vm:/tmp$ gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
Hello, world!
Jello, world!
```

图 37: 运行结果

3.7 实例七：使用 strace 跟踪系统调用

1、根据 strace 跟踪日志,ls -l 在执行时首先通过 `execve` 加载运行二进制程序 `/usr/bin/ls`,接着借助 `openat` 打开动态链接库、系统配置及当前目录;随后使用 `getdents64` 读取目录下的文件列表,并通过 `lstat/fstat` 获取每个文件的权限、所有者及修改时间等元数据;在此期间配合 `mmap/mprotect` 将所需库和文件映射到内存,最后通过 `write` 将格式化后的文件列表打印输出到标准输出,并用 `close` 关闭不再需要的文件描述符。

2、根据 `strace -e trace=openat` 的跟踪日志,Python 在启动并输出 `hello` 时主要打开了四类文件:首先是通过 `/lib/x86_64-linux-gnu/` 目录打开 C 运行时及基础系统功能的动态链接库;其次是打开系统的语言与时区配置文件;接着是打开 Python 解释器的核心标准库及预编译字节码文件;最后还打开了用于设置第三方扩展模块检索路径的配置文件。

3.8 实例八：通过 perf 对 C 语言采样并绘制火焰图

1、在终端创建一个 `slow.c` 文件,并写入如图所示代码

```
ouc@islouc-vm:/tmp$ cat << 'EOF' > slow.c
> #include <math.h>
> #include <stdio.h>
> double slow_computation(int n) {
>     double result = 0;
>     for (int i = 0; i < n; i++) {
>         for (int j = 0; j < 1000; j++) {
>             result += sin(i * j) * cos(i + j);
>         }
>     }
>     return result;
> }
> int main() {
>     double r = 0;
>     for (int i = 0; i < 100; i++) {
>         r += slow_computation(1000);
>     }
>     printf("Result: %f\n", r);
>     return 0;
> }
> EOF
```

图 38: slow.c

2、编译程序

```
ouc@islouc-vm:~$ gcc -g -O2 slow.c -o slow -lm
Terminal ouc-vm:~$ ls slow
SLOW
```

图 39: 编译程序

3、perf record 采样

```
ouc@islouc-vm:~$ perf record -g ./slow
WARNING: Kernel address maps (/proc/{kallsyms,modules}) are restricted,
check /proc/sys/kernel/kptr_restrict and /proc/sys/kernel/perf_event_paranoid.

Samples in kernel functions may not be resolved if a suitable vmlinux
file is not found in the buildid cache or in the vmlinux path.

Samples in kernel modules won't be resolved at all.

If some relocation was applied (e.g. kexec) symbols may be misresolved
even with a suitable vmlinux or kallsyms file.

Couldn't record kernel reference relocation symbol
Symbol resolution may be skewed if relocation was used (e.g. kexec).
Check /proc/kallsyms permission or run as root.
Result: -122140.186478
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.005 MB perf.data (58 samples) ]
```

图 40: 文件生成

4、perf report 查看热点

Samples: 58	of event	'cpu-clock:pppH',	Event	count (approx.): 14500000	
Children	Self	Command	Shared Object	Symbol	
+ 51.72%	51.72%	slow	libm-2.31.so	[.] __cos_fma	
+ 39.66%	39.66%	slow	libm-2.31.so	[.] __sin_fma	
+ 5.17%	5.17%	slow	slow	[.] slow_computation	
+ 1.72%	1.72%	slow	[unknown]	[k] 0xffffffff970f069d	
+ 1.72%	0.00%	slow	ld-2.31.so	[.] mmap64	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff97c00124	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff97b8d894	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff96e02999	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff96e49dd3	
+ 1.72%	1.72%	slow	[unknown]	[k] 0xffffffff970b650d	
+ 1.72%	0.00%	slow	libc-2.31.so	[.] __printf_chk	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff971082d5	
+ 1.72%	0.00%	slow	libc-2.31.so	[.] _IO_file_xsputn@@GLIBC_2.2.5	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff970dc4a8	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff9710b125	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff97c00c07	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff9710ad40	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff97b91e57	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff96ea2ba2	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff9710aa6c	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff97102328	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff97108f96	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff9710203c	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff97108a5d	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff970fea23	
+ 1.72%	0.00%	slow	[unknown]	[.] 0xffffffff97107630	

图 41: 各 CPU 占比

5、生成火焰图

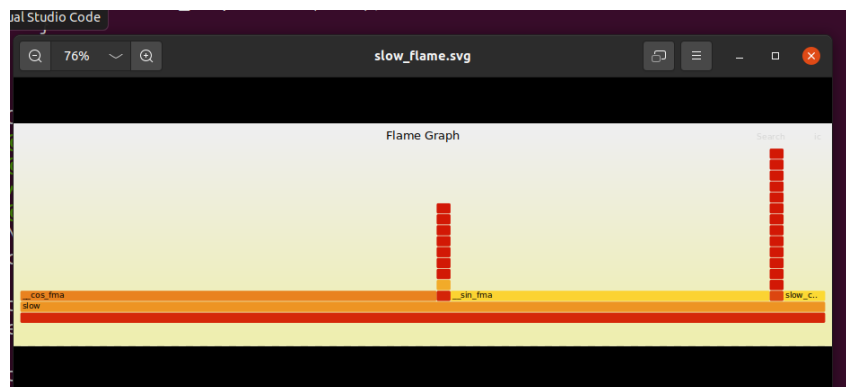


图 42: 火焰图

3.9 实例九：htop 监视 +taskset 绑定 CPU

1、安装工具并观察 CPU 核心数

```
ouc@islouc-vm:~$ sudo apt install htop stress taskset -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
E: Unable to locate package taskset
ouc@islouc-vm:~$ nproc
4
```

图 43: 结果截图

由截图可以看出有 4 个 CPU 核心。

2、打开 htop 监视，在终端输入 htop，htop 界面会实时显示每个 CPU 核心的占用率，要保持这个窗口开着

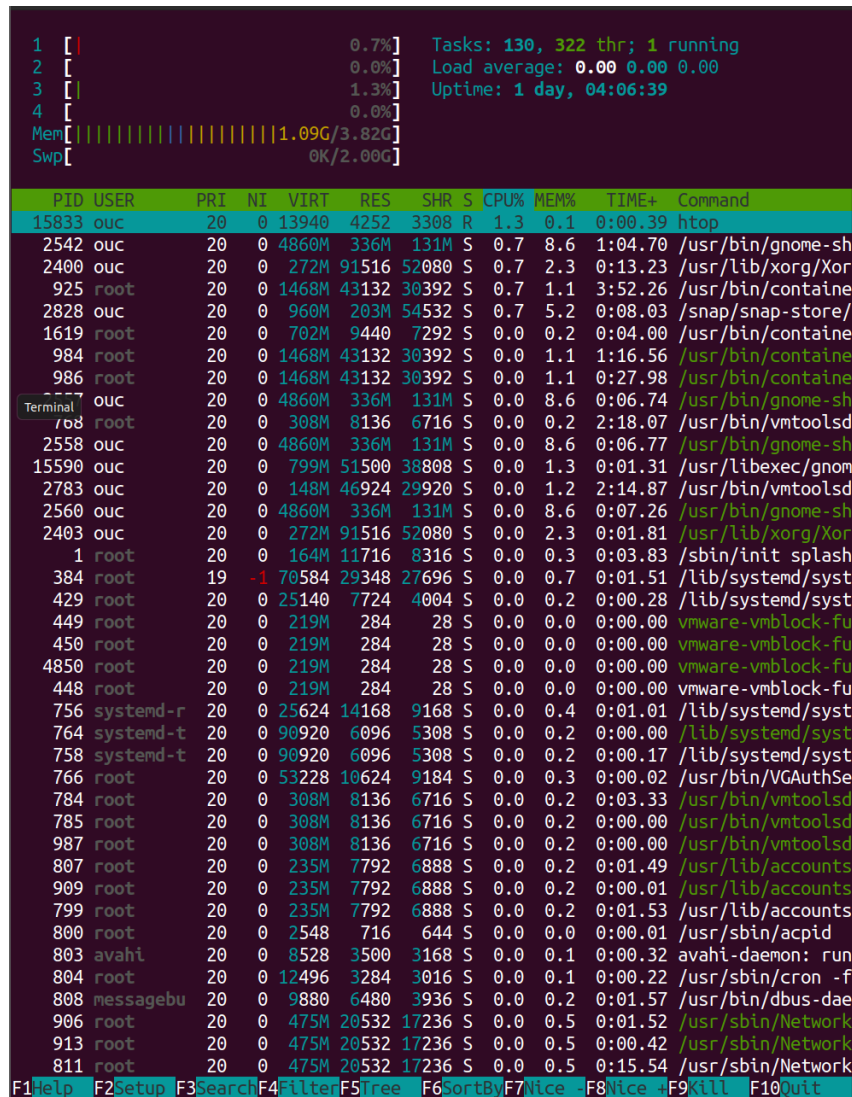


图 44: CPU 占用界面

3、另开一个终端，运行 stress 压测，回到 htop 窗口，由截图可以看出 3 个 CPU 核心跑满 100%，剩下一个是空闲的。

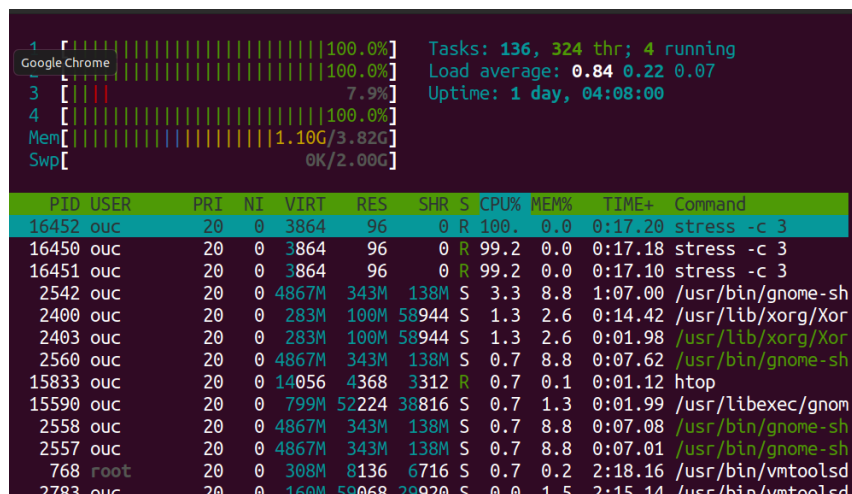


图 45: stress 压测截图

停掉 stress

4、用 taskset 限制 CPU，可以看出 taskset 只指定了 0 和 2。

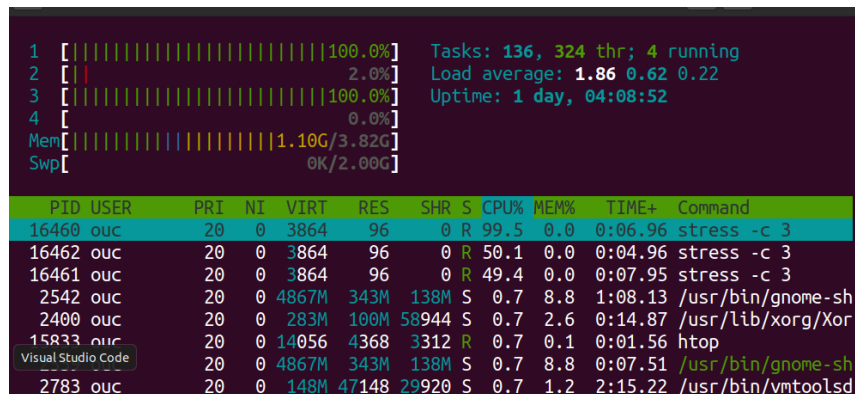
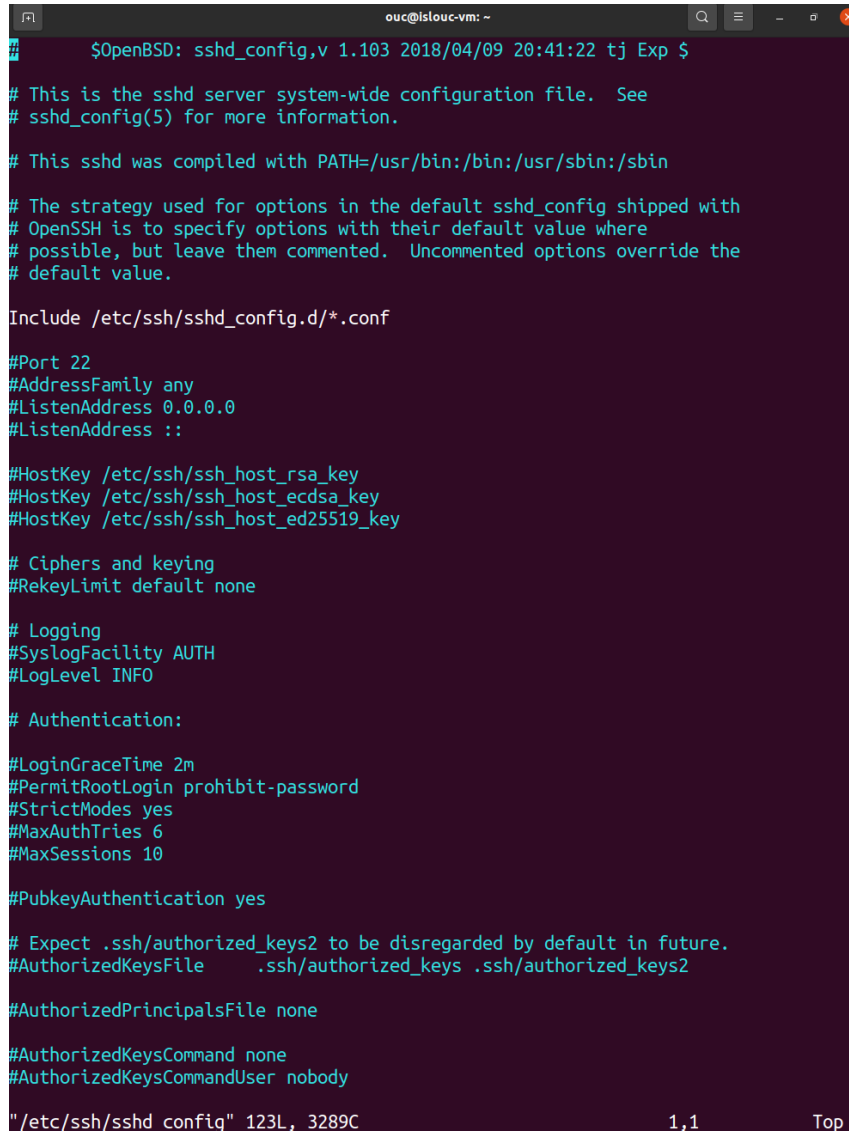


图 46: taskset 限制截图

5、为什么 stress 用了三个 CPU，但是 taskset 只指定了 0 和 2？答：taskset 只能把进程绑定到指定的 CPU 核心上，不能限制进程只能用几个核心。命令 taskset -cpu-list 0,2 stress -c 3 的含义是：stress -c 3：创建 3 个工作进程，每个进程都跑满一个 CPU；-cpu-list 0,2：把这 3 个进程都绑定到 CPU 0 和 CPU 2 这两个核心上。也就是说，taskset 限制的是能在哪些核心上跑。3 个进程会共享 0 和 2 这两个核心，总占用 3 个进程的负载会分摊到 2 个核心上。

3.10 实例十：修改 SSH 配置

1、编辑配置文件：sudo vim /etc/ssh/sshd_config,



```
$OpenBSD: sshd_config,v 1.103 2018/04/09 20:41:22 tj Exp $

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/bin:/bin:/usr/sbin:/sbin

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

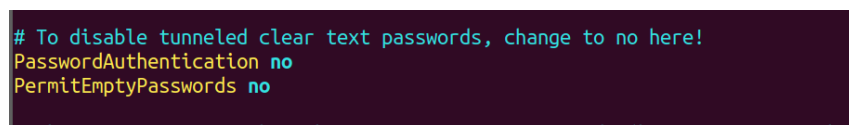
#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

"/etc/ssh/sshd config" 123L, 3289C 1,1 Top
```

图 47: 文件显示

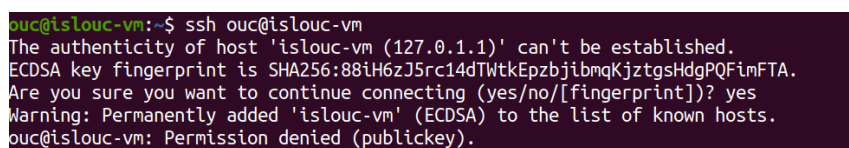
在 vim 中做如下操作：



```
# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication no
PermitEmptyPasswords no
```

图 48: 修改操作

2、重启 SSH 服务并测试 SSH 登录，可以看出输出 Permission denied (publickey) 说明，密码认证已成功禁用 SSH 服务重启成功。



```
ouc@islouc-vm:~$ ssh ouc@islouc-vm
The authenticity of host 'islouc-vm (127.0.1.1)' can't be established.
ECDSA key fingerprint is SHA256:88iH6zJ5rc14dTwtKEpzbjbmKjztgsHdgPQFimFTA.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'islouc-vm' (ECDSA) to the list of known hosts.
ouc@islouc-vm: Permission denied (publickey).
```

图 49: 结果截图

4 解题感悟

在此次实验中，我学习了命令行环境、开发工具调试、性能剖析与 C 语言内存错误排查等内容，练习了进程信号、代码重构、断点调试、性能采样优化等操作，我深刻认识到写代码不仅要实现功能，更要学会借助调试与分析工具发现隐患、优化效率，通过这些实验也加深了我对底层程序运行机制的理解。